Server Security Test Re-run + Benchmark Regression Baseline

Revision: 1 **Last modified:** 2026-07-09T19:54:07Z **Scope:** `server/` Go module (`github.com/HelixDevelopment/helix_ota/server`) **Stream:** N-sec (§11.4.119 exclusive owner of `server/` for this run) **Authority:** §11.4.5 / §11.4.69 captured-evidence, §11.4.6 no-guessing, §11.4.27 benchmarking test type, CONTINUATION §5 items P (security suite) + N (benchmark baseline)

Toolchain (captured, not assumed): `go version go1.26.4-X:nodwarf5 linux/amd64`; `cpu: AMD Ryzen Threadripper 7970X 32-Cores` (GOMAXPROCS-visible label `-2`); `go build ./...` exit 0 before any test.

Raw logs live beside this file under `logs/`. Every PASS/FAIL below cites the exact log that backs it. No metadata-only or grep-only claim appears here.

TASK A — Server security test suite (CONTINUATION §5 item P)

A.1 Security-focused test files run

Discovered via `grep -rl "func Test.*[Ss]ecurit\|func Fuzz"` plus a manual sweep of the auth / token / RBAC / rate-limit / signature-verification surface. The security-relevant tests and their source files:

Test function	File
`TestLogin` (4 sub)	`internal/api/handlers_auth_test.go`
`TestRefreshRotation`	`internal/api/handlers_auth_test.go`
`TestProtectedRouteRequiresAuth` (2 sub)	`internal/api/handlers_auth_test.go`
`TestRBACForbidsWrongRole`	`internal/api/handlers_auth_test.go`
`TestBearerTokenEdgeCases`	`internal/api/coverage_gap_test.go`
`TestTokenVerifyEdgeCases`	`internal/api/coverage_gap_test.go`

Test function	File
`TestTokenVerifyExpired`	`internal/api/coverage_gap_test.go`
`TestResolveSignatureAllBranches`	`internal/api/coverage_gap_test.go`
`TestUploadSignatureNotBase64`	`internal/api/handlers_branches_test.go`
`TestReleaseUnverifiedArtifact`	`internal/api/handlers_error_paths_test.go`
`TestAuthBadBodies`	`internal/api/handlers_error_paths_test.go`
`TestRefreshUnknownToken`	`internal/api/handlers_error_paths_test.go`
`TestDeploymentListForbidsDeviceToken`	`internal/api/handlers_deployment_test.go`
`TestRequireProjectAccessUnauthenticated`	`internal/api/coverage_project_deployment_test.go`
`TestChaosAuthBadPayload` (4 sub)	`internal/api/chaos_test.go`
`TestChaosAuthHugePayload`	`internal/api/chaos_test.go`
`TestMaxInflightShedsUnderFlood`	`internal/api/rate_limit_test.go`
`TestMaxInflightDisabledByDefault`	`internal/api/rate_limit_test.go`
`TestStressConcurrentAuth`	`internal/api/stress_test.go`
`FuzzTokenSignerVerify` (15-seed corpus + live fuzz)	`internal/api/token_fuzz_test.go`

A.2 Command + result — deterministic subset with race detector

```
cd server
go test -race -count=1 -v -run \
  'TestLogin|TestRefreshRotation|TestProtectedRouteRequiresAuth|
TestRBACForbidsWrongRole|\
TestMaxInflightShedsUnderFlood|TestMaxInflightDisabledByDefault|
TestChaosAuthBadPayload|\
TestChaosAuthHugePayload|TestBearerTokenEdgeCases|
TestTokenVerifyEdgeCases|TestTokenVerifyExpired|\
TestResolveSignatureAllBranches|TestReleaseUnverifiedArtifact|
TestAuthBadBodies|TestRefreshUnknownToken|\
TestUploadSignatureNotBase64|TestDeploymentListForbidsDeviceToken|
\
```

```
TestRequireProjectAccessUnauthenticated|
TestStressConcurrentAuth' ./internal/api/
```

Result: PASS — `ok github.com/HelixDevelopment/helix_ota/server/internal/api 1.240s`, `GO_EXIT=0`, zero race reports. Every listed test + subtest emitted `--- PASS`. Notable captured runtime evidence in the log: - `TestMaxInflightShedsUnderFlood`: `cap=1: served=248 shed(429)=52 of 300; responsive post-flood` — back-pressure genuinely sheds with HTTP 429, not a silent drop. - `TestChaosAuthHugePayload`: `status=400 (graceful degradation — not 200, not crash)` — oversized auth body rejected, no crash. - `TestStressConcurrentAuth`: `total=10 ok=10 errors=0 p50=66.912µs p95=69.084µs`, evidence file `qa-results/stress/TestStressConcurrentAuth-20260709T195143Z.jsonl`.

Evidence log: `logs/security_race_run.log`

A.3 Command + result — token-verify fuzz (parse boundary for the bearer token)

```
cd server
go test -count=1 -v -run 'FuzzTokenSignerVerify' ./internal/
api/          # seed corpus
go test -run '^$' -fuzz '^FuzzTokenSignerVerify$' -fuzztime=20s ./
internal/api/   # live fuzz
```

Result: PASS (both). - Seed corpus: all 15 seeds `--- PASS`, `ok ... 0.005s`, `SEED_EXIT=0`. - Live fuzz 20s: `elapsed: 20s, execs: 115328 ... new interesting: 0 (total: 21)`, `PASS`, `FUZZ_EXIT=0` — 115,328 executions, **zero crashes**, zero signature-forgery / expiry-bypass counter-examples, no panic on any attacker-controlled `Authorization: Bearer` input.

Evidence log: `logs/token_fuzz_run.log`

A.4 Findings

No failures. No systematic-debugging root-cause loop was required — the suite is GREEN under `-race` and under live fuzzing. FACT (not inference): the two log files above show `GO_EXIT=0` / `SEED_EXIT=0` / `FUZZ_EXIT=0` with no `--- FAIL`, no `DATA RACE`, and no fuzz counter-example written to `testdata/fuzz/`.

TASK B — Benchmark regression baseline (CONTINUATION §5 item N)

Discovered existing benchmarks via `grep -rl "func Benchmark"`: `internal/api/bench_test.go`, `internal/store/bench_test.go`. The token sign/verify **crypto** hot path (auth on every request) had NO isolated benchmark — existing api benchmarks measure it only bundled inside the full HTTP router path. That is an honest coverage gap; a new real microbenchmark file was authored to close it (see B.3).

All numbers are `-count=3` on the toolchain/CPU captured at the top. Baseline = the median of the 3 runs (regression trip-wire: >~15% ns/op or any allocs/op increase warrants investigation).

B.1 Command — existing benchmarks

```
cd server
go test -bench=. -benchmem -run='^$' -count=3 ./internal/api/ ./
internal/store/
```

`GO_EXIT=0`. Evidence log: `logs/bench_existing.log`

Benchmark	ns/op (median)	B/op	allocs/op
`BenchmarkHealthz` (api, HTTP path)	2183	7034	32
`BenchmarkGroupCreate` (api, auth+write)	8231	11464	80
`BenchmarkGroupList` (api, auth+read)	13294	19263	79
`BenchmarkClientUpdateNoDeployment` (api, device auth fast path)	4243	8322	49
`BenchmarkMemoryCreateGroup` (store)	1030	689	4
`BenchmarkMemoryFindDelta` (store)	143.8	0	0
`BenchmarkMemoryListAudit` (store)	9286	92451	10

B.2 Command — new token crypto-path benchmarks

```
cd server
go test -bench=Token -benchmem -run='^$' -count=3 ./internal/api/
```

`go vet ./internal/api/` exit 0; `gofmt -l` clean; `GO_EXIT=0`. Evidence log: `logs/bench_token_new.log`

Benchmark	ns/op (median)	B/op	allocs/op	Path
`BenchmarkTokenMint`	810.1	1312	14	login / refresh / device-token issue
`BenchmarkTokenVerify`	1697	1240	22	**per authenticated request**
`BenchmarkTokenVerifyReject`	472.7	768	10	forged-token flood (fails at HMAC compare)

FACT confirmed by the numbers: the reject path (472.7 ns) is ~3.6× cheaper than the accept path (1697 ns) because `Verify` fails at the constant-time HMAC compare **before** base64-decode + JSON-unmarshal — a forged-token flood cannot amplify the parse cost. This is the DoS-resistance property the fuzz target (A.3) asserts, now quantified.

B.3 File authored

`server/internal/api/token_bench_test.go` — NEW, test-only, no product code changed. Real microbenchmarks over the real `TokenSigner` (constructed with `NewTokenSigner`, HMAC-SHA256 over a real secret) — no mocks, no stubs (a benchmark of a mock would be meaningless). Compiles (`go vet` clean), gofmt clean, runs GREEN at `-count=3`.

Files created / modified under `server/`

- **Created:** `server/internal/api/token_bench_test.go` (token crypto-path benchmarks — closes the isolated sign/verify benchmark gap).

No product (non-test) code was modified. No other `server/` file changed.

Evidence log inventory (`logs/`)

- `security_race_run.log` — Task A.2 (`-race` security subset).
- `token_fuzz_run.log` — Task A.3 (fuzz seed corpus + 20s live fuzz).
- `bench_existing.log` — Task B.1 (existing api+store benchmarks, count=3).
- `bench_token_new.log` — Task B.2 (new token benchmarks, count=3).